Department of Computer Science & Engineering
The University of Texas at Arlington

Project Charter
CSE 4316: Senior Design I
Fall 2025

ProtoBot
StarCraft Brood War Game Playing Agent

David Daffern
Marshall Helton
Aadhitya Kumar
Preston Loera
Matthew Nguyen
Tung Nguyen

Revision History

Revision	Date	Author(s)	Description
0.1	09.12.2025	PL, MN	document creation
0.2	09.26.2025	PL, MN	complete draft
1.0	03.23.2026	PL	Updated project charter
2.0	04.10.2026	PL, MN	Revisions to updated project charter

Contents

1 Problem Statement **6**

2 Methodology **6**

3 Value Proposition **6**

4 Development Milestones **7**

5 Background **7**

6 Related Work **8**

7 System Overview **9**

8 Roles & Responsibilities **10**

9 Cost Proposal **10**
- 9.1 Preliminary Budget . . . 10
- 9.2 Current & Pending Support . . . 11

10 Facilities & Equipment **11**

11 Assumptions **12**

12 Constraints **12**

13 Risks **14**

14 Documentation & Reporting **14**
- 14.1 Major Documentation Deliverables . . . 14
  - 14.1.1 Project Charter . . . 14
  - 14.1.2 System Requirements Specification . . . 14
  - 14.1.3 Architectural Design Specification . . . 14
  - 14.1.4 Detailed Design Specification . . . 14
- 14.2 Recurring Sprint Items . . . 14
  - 14.2.1 Product Backlog . . . 14
  - 14.2.2 Sprint Planning . . . 14
  - 14.2.3 Sprint Goal . . . 15
  - 14.2.4 Sprint Backlog . . . 15
  - 14.2.5 Task Breakdown . . . 15
  - 14.2.6 Sprint Burn Down Charts . . . 15
  - 14.2.7 Sprint Retrospective . . . 15
  - 14.2.8 Individual Status Reports . . . 15
- 14.3 Closeout Materials . . . 16
  - 14.3.1 System Prototype . . . 16
  - 14.3.2 Project Poster . . . 16
  - 14.3.3 Web Page . . . 16
  - 14.3.4 Demo Video . . . 16

14.3.5 Source Code 16
14.3.6 Source Code Documentation 17
14.3.7 Installation Scripts 17
14.3.8 User Manual 17

List of Figures

1 High Level ProtoBot Design 10
2 Burn-down chart from after Sprint 1 15

1 Problem Statement

StarCraft: Brood War (1998) is a real-time strategy game developed by Blizzard Entertainment and has been used as a means for AI research for more than a decade now. Due to the game's real time environment and mechanics such as the fog-of-war (where map visibility is limited to a radius around units), this creates difficult problems for artificial intelligence algorithms to solve even with modern computing and AI approaches. Many competitions and ranked bot ladders have been made for authors to create their own bots and play against each other, all with the goal of advancing artificial intelligence methods.

2 Methodology

Our team will develop a *StarCraft: Brood War* game-playing agent with the goal of competing in the COG competition and uploading our bot to an online ranking ladder BASIL. ProtoBot will use a variety of artificial intelligence algorithms to address tasks such as path finding, resource management, unit manipulation, etc. The team will incorporate rule-based algorithms to be able to solve these problems and interface with *StarCraft: Brood War* to win the most games possible using intelligent play. ProtoBot will accomplish all these tasks through a modular structure in which each task is handled by a component of the bot. By separating tasks into individual components, these components can simplify the problem space for specific tasks and help communicate game information.

ProtoBot will be able to use algorithms to optimally:

- Assign workers to resources and manage them accordingly.
- Follow a given build order for units and buildings early on.
- Track enemy players through unit scouting and location memorization.
- Adapt unit production to ongoing game information and enemy unit compositions.
- Use path-finding algorithms to traverse the map and position units appropriately.
- React to enemy movements and abilities either evasively or defensively.
- Expand both its base and its economy throughout the course of a game.
- Recognize when to attempt to end the game rather than continuously grow.

While there are tournaments that disable the ”Fog-of-War”, both COG and BASIL have the ”Fog-of-War” enabled in their rules, meaning that ProtoBot's AI algorithms will take into account the game's natural "Fog-of-War" and operate under imperfect information.

3 Value Proposition

Our agent, ProtoBot, would provide significant value to the *StarCraft: Brood War* AI community by contributing a bot that is capable of playing games against other bots and exposing UTA students to the StarCraft AI community. UTA benefits from supporting a project that allows future senior design teams to be able to explore other games besides *StarCraft: Brood War* as a means for a senior design topic. Other games, such as *StarCraft II*, *Age of Empires*, *Beyond All Reason*, *Dota 2*, *League of Legends*, etc., all provide a means for AI research and allow students to have the opportunity to address a challenging problem utilizing the courses provided by UTA. Future teams wanting to make a game-playing agent would also benefit from this project since ProtoBot gives teams access to resources and insights of a team that successfully made one.

4 Development Milestones

The following list provides milestones and estimated completion dates for the project.

- Project Charter first draft - September 2025
- System Requirements Specification - October 2025
- Architectural Design Specification - November 2025
- Demonstration of Rule-Based bot Architecture - December 2025
- Detailed Design Specification - March 2026
- CoE Innovation Day poster presentation - May 2025
- Demonstration of Finalized bot with statistics of ProtoBots performance - May 2025
- Final Project Demonstration - May 2025

5 Background

For our sponsors, in this case UTA, ProtoBot represents the first steps toward establishing game-playing agents as a meaningful senior design project topic. This type of project allows students to tackle a challenging problem that trains both their problem-solving and programming skills. Through ProtoBot, our aim is to demonstrate how rule-based methods can be used to simplify and optimize the decisions that a game requires a player to make. In addition, the project also demonstrates how an agent can be structured to retain and utilize the information gathered over the course of a game. If time permits the team will attempt to incorporate machine learning methods in ProtoBot to improve rule based approaches.

6 Related Work

Since 2011, more than 100 StarCraft bots have been developed with some of the oldest bots being around fifteen years old. Each bot has their own unique methods to solve the problems StarCraft presents, but generally follow the same structure [8]. StarCraft has also been used for academic purposes with solutions to problems such as scheduling, resource management, and placement of buildings being a sample of the many papers published today.

Additionally, thanks to research done by the StarCraft AI community, some of the problems StarCraft presents have been mostly solved with multiple "good enough" solutions for community members to use. Examples include A* with influence map pathfinding [3], mineral optimization [1], and wall building [7].

Google has presented its own agents for *StarCraft II* called AlphaStar, but there has been a debate in the community as to whether or not the bot achieved the goals it set out to reach. While AlphaStar defeated some professional players, it relied heavily on methods that only a computer can use, such as action banking, which took advantage of AlphaStar's total action calculation the bot used over to display its actions per minute [4]. Saving in-human action performance for large fights.

Multiple competitions require that developers who use a preexisting bot must substantially modify its code structure for the bot to be admitted to competitions. Therefore, we have opted to develop a bot from scratch. Another limitation of using existing solutions is the certain bots only work on specific map settings that remove the fog-of-war, where many competitions require fog to be enabled. Additionally, The age of a bot affects the quality of the actions [6], as older bots have not been implemented on for many years and are often being used a benchmark for new bots.

This is also where our team believes we can attempt to improve upon the solutions presented in these papers. With SPARCraft for UAlberta bot being able to manipulate 32v32 combat situations [2], using Alpha-Beta Considering Durations to arrive at an okay answer that achieves a better result based on how long you want to wait for an answer. The reason we would not want to use this method is because of the computational limit of 42 ms for COG. Instead, by focusing our computational time into other modules such as scouting and strategy, we can make combat simpler, we can manipulate units as a group to save on computational costs and abstract the methods required to manipulate large amounts of units.

There also seems to be no sophisticated way to be able to scout similar to professional players, with the only research found by our team being PurpleWave using a state machine to scout and many other similar methods [5]. This is where our team can do the most research if time permits, by creating a sophisticated rule based algorithm that can then use machine learning to move similar to professional players. Our running theory is that by putting our time and effort into creating a scouting algorithm that minimizes the amount of unknowns, we can arrive at a most likely answer due to not missing as much information compared to other bots [9].

7 System Overview

The major components of our agent will consist of the Combat Manager, Scouting Manager, Economy Manager, Build Manager, Strategy Manager, Information Manager, and Map Tools.

Combat Manager: The primary function of the combat manager is the micro management of combat units in defensive and offensive scenarios. The Combat Manager will take in information from multiple sources, like the Scouting Manager and the influence map, to be able to move units around to attack weak spots in the opponent's defenses, maneuver units away from enemy units that are waiting on their attack cooldown's, and keep units that are weak against other units away from them.

Scouting Manager: The primary function of the scouting manager is to provide multiple modules with the necessary information to react to enemy moves. During the early portions of the game, the module will utilize probes to gather necessary information. This area of the bot is where we will provide research as to how to create a scouting AI that can perform similarly to professional players.

Economy Manager: The primary function of the economy manager is to manipulate our workers to farm the essential resources to build units that can effectively counter the opponent's units. ProtoBot will use a rule-based approach for this module to save on computational time. Mineral farming has been researched and optimized in the *StarCraft: Brood War* AI field, so our bot should be able to perform similarly.

Build Manager: The primary function of the build manager during the early stages of the bot is to use hard-coded build orders to build the necessary buildings at the right build power to execute a strategy. Additionally, the build manager is responsible for finding suitable locations to place down buildings.

Strategy Manager: The strategy manager's primary function is to decide on the best course of action based on the information gathered. The information manager provides the strategy manager with information on how the enemy is playing the game, and the strategy manager will come up with the most likely answer to combat the enemy. Strategy will control the macro of any squads ProtoBot has and control the production of units and buildings to be as efficient as possible.

Information Manager: The information manager is the main way information about the enemy is stored and used in the rest of our bot. The information should give a general sense of how strong the enemy is, the types of units they are building, where units are, and what style of game play the enemy is playing. This class will be accesible to all modules and be created as a static instance.

Map Tools: Map Tool is the class provided by StartBot for general use and is mostly used for debugging.

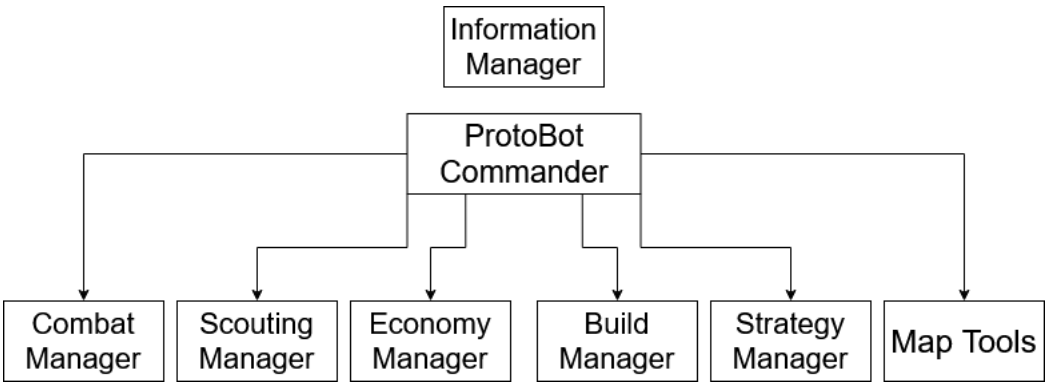


Figure 1: High Level ProtoBot Design

8 Roles & Responsibilities

Product Lead: Preston Loera
SCRUM Master: Matthew Nguyen
The Team: David Daffern, Marshall Helton, Aadhitya Kumar, and Tung Nguyen

These are the current roles that have been chosen by the team, but are subject to change as needed for the successful development of this project. These roles are to be permanent, with each role having the following responsibilities.

The Product Owner will be the primary point of communication between project owner and the team, reaching out when required. The Product Owner will prioritize and adjust features being implemented in the project as needed.
The SCRUM Master will be responsible for ensuring SCRUM deadlines are met on time, ensuring the team is functioning properly and efficiently. This includes communicating with all team members to Make sure they are successfully fulfilling all of their required responsibilities on JIRA and communicating sprint deadlines through scheduled meetings/Discord.
Everyone on the team, including the Product Lead and SCRUM Master, is responsible for the development of the project. The team will meet in person for the first few sprints to develop the architecture of our agent, and will become self-organizing after development has started. During development, the team will meet and work on the project in person as needed to achieve sprint requirements.

9 Cost Proposal

Normally, a team geared towards AI software development would need to pay for API calls to an AI-model if they weren't making one in-house. However, development of ProtoBot will be done through an open-source API (BWAPI) and the main focus will be the AI and ML algorithms used by ProtoBot for its various tasks in the game. Therefore, there will rarely be any expenses for the development of ProtoBot's software. However, there could be a small possibility that we could need software licenses for UML diagrams or project organization, so it is best to be prepared for any unexpected costs.

9.1 Preliminary Budget

Items	Expected Cost
Unexpected Costs	100

9.2 Current & Pending Support

As of now, there are no current or pending funds for the development of ProtoBot.

10 Facilities & Equipment

The creation of ProtoBot will be a software development focused project, so there is no need for specific lab spaces to complete this project. The open source API for StarCraft Brood War, BWAPI, gives us all the foundational tools needed to start on ProtoBot. The rest of the program, mainly the algorithms, will be coded on our own devices in C++ using common programming software such as GitHub, Visual Studio, etc.

11 Assumptions

The following list contains critical assumptions related to the implementation and testing of the project.

- BWAPI will allow ProtoBot to have access to all game information and mechanics that a player would
- A very early version of ProtoBot with minimal implementations will be done by the end of Fall for further optimizing in Spring
- No other software will be needed for ProtoBot's development besides StarCraft APIs and our own code
- Some way to speed up testing will be found as we research further. Testing is a big issue as some components only affect gameplay during the middle of a match and matches can last as long as 20 minutes.
- Rule-based algorithms implemented during the Fall will be able to be improved during the Spring.

12 Constraints

The following list contains key constraints related to the implementation and testing of the project.

- Final prototype demonstration must be completed by May 1st, 2025.
- If no competition is available for registration during the spring, the team will benchmark Protobot's performance using the online ranking ladder BASIL that uses the SSCAIT ruleset.
- AIIDE Ruleset:
  - Bots onframe call does does not exceed limit of 42ms.
  - >= 1 frames exceed 10 seconds, or
    >= 10 frames exceed 1 second, or
    >= 320 frames exceed 55ms
  - Benchmarking games will have a time limit of 86400 frames.
  - Benchmark games will be 1 vs. 1 full game StarCraft: BroodWar 1.16.1 with fog of war enabled.
  - All games will be played at setLocalSpeed(0) (fastest setting).
  - Bots will have read access to folder 'bwapi-data/read/' and write access to folder 'bwapi-data/write/', both of which will be in the standard location under the StarCraft root directory.
- BASIL Ruleset:
  - **Ranking:**
    - Bots are assigned ranks similar to those in SC:R.
    - Within a rank, the bot's position is determined by its ELO.
    - If a bot climbs or drops in rank, it is locked in that rank for the next 200 games to prevent hysteresis and allow adaptation to new challenges. New bots are UNRANKED for their first 200 games.
    - Ranks are updated once per day.

- **Games:**
  - All games are 1v1.
  - *StarCraft: Brood War* version 1.16.1 is used.
  - No time limit per frame due to technical limitations.
  - Real-time limit: 30 minutes; if exceeded, the game is killed and no win/loss is assigned. Bots causing this frequently may be disabled; intentionally causing timeout may result in a ban.
  - Game-time limit: 60 minutes; the player with the highest in-game kills + razings score wins. If tied, the real-time limit rule applies.
  - Draws occur only if bots have identical scores and time per frame (rare) and are not counted as wins/losses.
  - Speed setting: LF3 (Normal).
  - Map pool: SSCAIT.
  - Random race is not possible; random bots are assigned a race before the game.
  - A crash counts as a loss unless both bots crash.
  - All played games are counted as "played," even if a crash occurs.
- **Bots:**
  - Standard SSCAIT rules apply.
  - `bwapi-data/write` will be copied to `bwapi-data/read`. By default, the read directory is not cleared when a bot is updated.
  - Bots are automatically updated every 6 hours.
  - No other bot author can view or download your read/write directories.
  - All replays, game logs, bot logs, and crash logs are publicly downloadable.
  - Internet/network access is generally prohibited (exceptions apply).
  - Hardware resources:
    - 1 CPU core of AMD Ryzen 7 1700X.
    - 1 GB RAM (excluding OS usage).
    - Up to 100 MB disk space for read/write/ai directories.
  - Special options (add as `BASIL: <option>[,<option>...]`):
    - RESET - Clears the directory on the next update (first update only).
    - PB-KEY-<KeyId>- Publish the read directory compressed and encrypted using the given key.
    - PUBLISH-READ - Publish the read directory compressed but not encrypted.
    - MAP-POOL:;... - Add support for additional map pools.
  - Bots are disabled if:
    - Requested by the author.
    - Win-rate falls below 15% in the last 100 games. Updating the bot will re-enable it, and only games since the update will count.

13 Risks

The following high-level risk census contains identified project risks with the highest exposure. Mitigation strategies will be discussed in future planning sessions.

Risk description	Probability	Loss (days)	Exposure (days)
Developer(s) becomes sick and cannot work on project	0.10	5	0.5
Agent complexity makes latency exceed tournament standard (42ms AIIDE or BASIL 82ms)	0.40	10	4
Unexpected bugs with BWAPI affecting ProtoBot development	0.20	5	1
Delays in component development affecting other components	0.20	7	1.4
Changes/optimizations in BWAPI occur during ProtoBot development	0.10	5	0.5

Table 1: Overview of highest exposure project risks

14 Documentation & Reporting

14.1 Major Documentation Deliverables

14.1.1 Project Charter

The Project Charter will be maintained and updated throughout the Fall and Spring semesters. This document will be reevaluated after every lab meeting, which will occur every Friday from 1:00 pm to 1:50 pm but could extend to 2:50 pm. The initial version will be delivered on 9/22/2025. The final version will be submitted during the Spring semester after any changes are made to the document.

14.1.2 System Requirements Specification

The System Requirements Specification will be maintained and updated throughout the Fall and Spring semesters. This document will be reevaluated after every lab meeting, which will occur every Friday from 1:00 pm to 1:50 pm but could extend to 2:50 pm. The initial version will be delivered on 10/17/2025. The final version will be submitted during the Spring semester after any changes are made to the document.

14.1.3 Architectural Design Specification

The System Requirements Specification will be maintained and updated throughout the Fall and Spring semesters. This document will be reevaluated after every lab meeting, which will occur every Friday from 1:00 pm to 1:50 pm but could extend to 2:50 pm. The initial version will be delivered on 11/7/2025. The final version will be submitted during the Spring semester after any changes are made to the document.

14.1.4 Detailed Design Specification

This document has not been published in Canvas yet because we will be working on it during Senior Design II. This section will be updated once we begin creating the detailed design specification.

14.2 Recurring Sprint Items

14.2.1 Product Backlog

Items will be added to the product backlog from the SRS from highest priority to lowest priority. The priority of these items will be decided by group discussion and voting. To maintain and share product backlog, we will use a free account through Jira, a project organization software.

14.2.2 Sprint Planning

During the last week of a sprint, we will discuss the plan for the next spring during our lab meeting on Friday. In total, there will be 4 sprints with each one lasting 3 weeks.

14.2.3 Sprint Goal

The sprint goal will initially be stated by the product owner and then possibly changed after a group discussion with the scrum master and developers. Because this project is not sponsored, there will be no customer involved in ProtoBot's development.

14.2.4 Sprint Backlog

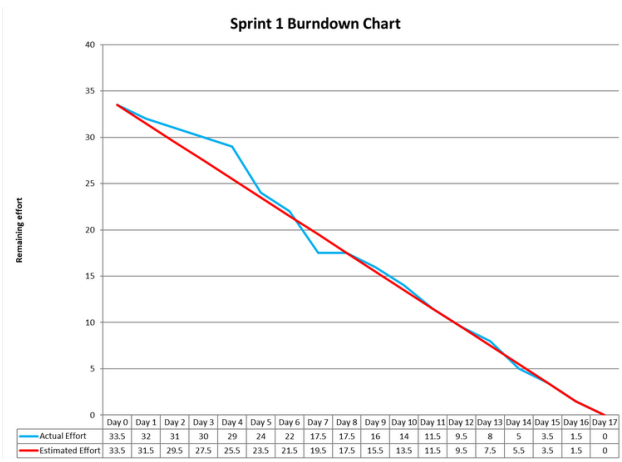
The product owner and the scrum master will discuss which product backlog items will enter the sprint backlog. The sprint backlog will be maintained along with the product backlog in Jira.

14.2.5 Task Breakdown

Currently, each team member will voluntarily claim a task, with unclaimed tasks going to team members who did not volunteer to do so. After every task is complete by a member, they will put their actual time taken on the item in Jira. That way, we can have a comparison between the estimated time taken and actual time taken.

14.2.6 Sprint Burn Down Charts

The scrum master will be responsible for generating the burn-down charts for each sprint. Toward the end of the sprint, the scrum master will look at completed items in the sprint backlog and total the hours it took to complete them. For incomplete tasks, the scrum master will contact the developer assigned to that task to ask for their time. The burn-down chart will be a simple graph with the x-axis as the days of the sprint and the y-axis as the hours left. The graph will have two lines, one being for the estimated hours and the other for the actual hours spent on that day.



	Day 0	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7	Day 8	Day 9	Day 10	Day 11	Day 12	Day 13	Day 14	Day 15	Day 16	Day 17
Actual Effort	33.5	32	31	30	29	24	22	17.5	17.5	16	14	11.5	9.5	8	5	3.5	1.5	0
Estimated Effort	33.5	31.5	29.5	27.5	25.5	23.5	21.5	19.5	17.5	15.5	13.5	11.5	9.5	7.5	5.5	3.5	1.5	0

Figure 2: Burn-down chart from after Sprint 1

14.2.7 Sprint Retrospective

The sprint retrospective will be discussed as a group during our lab meeting on the last week of the sprint. The sprint retrospective will be discussed before planning the next sprint on the same day. The product/sprint backlogs will be updated on Jira. Each member will document their individual reports throughout the last week and submit them by the end of Friday.

14.2.8 Individual Status Reports

Each individual member will report their status on the tasks that they volunteered for or were assigned. This report will be done at the last week of every sprint (3 weeks). Some key items in the report will be

the tasks that they completed, how long it took to complete them, and any incomplete tasks that need to be moved to the next sprint.

14.3 Closeout Materials

14.3.1 System Prototype

In the final system prototype, we will include a working game-playing agent that can complete a full match of *StarCraft: Brood War* against a bot or player, albeit sub-optimally. The system prototype will be revealed at the end of the semester during demo day on 12/03/2025. There will be no prototype/field acceptance test.

14.3.2 Project Poster

The poster will include the diagram/structure of ProtoBot. It will also have the AI/ML algorithms used and how these algorithms contribute to the efficiency of ProtoBot. The poster size will be 36" x 48" and the poster will be delivered at the end of the semester alongside the demo.

14.3.3 Web Page

The project web page will include:

- Team name
- Timeline
- Team members
- Abstract
- Background
- Project Requirements
- Design Constraints
- Schedule Constraints
- Key Engineering Standards
- System Overview
- Video Functionality
- Future Work

14.3.4 Demo Video

The demo video will talk about ProtoBot's functionality. Describing a high level overview of how ProtoBot makes decisions and any current issues with ProtoBot that will be worked on during Senior Design II in the Spring. The demo video will be at least 5 minutes long.

14.3.5 Source Code

The source code for ProtoBot will be maintained through GitHub. Since our project has no customer, our project's source code will be kept visible on GitHub for others to critique or gain insight from. Our GitHub repository will have an MIT license so others can make and distribute closed source versions if they want to.

14.3.6 Source Code Documentation

The documentation will be as clear and concise as possible so that users and other programmers can quickly understand each function or class's purpose. To generate documentation, we will be using Doxygen, which is a documentation software that works well with c++ code. The final documentation will be provided in an HTML format.

14.3.7 Installation Scripts

The ProtoBot GitHub repository will include BWAPI. To get *StarCraft: Brood War*, we will give a link to the StartCraft repository, made by Dave Churchill. This is because Dave Churchill runs the AIIDE StarCraft AI Competition and was given permission by Blizzard, the creators of StarCraft Brood War, to provide the game files in his repository for others to use. The only installations needed will be the ProtoBot GitHub files and the *StarCraft: Brood War* file with the BWAPI data files.

14.3.8 User Manual

The project will include a digital user manual with instructions on how to get ProtoBot working with *StarCraft: Brood War* 1.16.1.

References

[1] D. B. Christensen, H. O. Hansen, L. Juul-Jensen, and K. Kastaniegaard. Efficient resource management in starcraft: Brood war. 2010.

[2] David Churchill, Abdallah Saffidine, and Michael Buro. Fast heuristic search for rts game combat scenarios. In *Proceedings of the Eighth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, AIIDE 12. AAAI Press, 2012.

[3] Lucas Critch and David Churchill. Sneak-attacks in starcraft using influence maps with heuristic search. In *2021 IEEE Conference on Games (CoG)*, pages 1–8, 2021.

[4] DeepMind. Alphastar: Master agent in starcraft ii, 2019.

[5] Dgant. Dgant/purplewave: Starcraft: Brood war ai in scala.

[6] Santiago Ontanon, Gabriel Synnaeve, Alberto Uriarte, Florian Richoux, David Churchill, and Mike Preuss. A survey of real-time strategy game ai research and competition in starcraft. *IEEE Transactions on Computational Intelligence and AI in Games*, 2013.

[7] Martin L. M. Rooijackers and Mark H. M. Winands. Wall building in the game of starcraft with terrain considerations. In *2018 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–5, 2018.

[8] TheEngineeringBay. Awesome broodwar resources, 2023.

[9] P. Warr. Starcraft: Building a brilliant brood war bot, 2023.